

Dense Graphs, Fast Traversals: Benchmarking SQL vs Graph Engines at Scale

Hirishikesh Parthasarathy



Master of Science
School of Informatics
University of Edinburgh
2025

Abstract

This research examines how graph density and size influence the speed of traversal and path-checking queries across various graph-processing methods. We generated synthetic Erdős–Rényi graphs, treating them as undirected instances. The queries involve reachability tasks and path-checking tasks under Cypher or recursive SQL. We tested two density conditions: constant edge probability, p (constant density), and scaling probability $p(N) = N^{-a}$ with $a \in (0, 1)$. Our experiments included node counts $N \in \{100, 500, 1000, 2000, 5000\}$ and various density values. We explored three approaches to graph processing: using NetworkX for a basic in-memory setup, employing native graph database engines (Memgraph and Neo4j) with Cypher, and using recursive SQL in relational databases (PostgreSQL and DuckDB). For each method, we tracked data import time, query time, the number of retrieved paths, and details of the query plan. The experiments included several trials, excluding initial warm-up runs, and the results are presented as means with 95% confidence intervals.

The results show that higher density leads to slower traversal speeds. Import and check times increase more quickly than edge count, and dense graphs return more paths while requiring more computation. Although these queries are polynomial-time in theory, dense graphs cause a notable increase due to large intermediate matches. Graph database engines using Cypher perform best for multi-hop traversal when graphs are sparse to moderately dense. Relational recursive-CTE methods still work well for shallow or set-oriented questions. This work offers: (1) a repeatable experimental setup and data, (2) an empirical view of performance based on N and density, and (3) practical advice on selecting graph-processing architectures based on query type and density. All scripts and data are available for reproducing the results.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(Hirishikesh Parthasarathy)

Acknowledgements

First, I want to express my deepest gratitude to my supervisor, Professor Leonid Libkin. His guidance, encouragement, and careful proofreading have been invaluable at every stage of this dissertation. Despite his many commitments, Professor Libkin always found time to offer insights and support, helping me refine and strengthen my work.

I am also very thankful to my DLSS tutor, Jane, whose organizational skills and thoughtful feedback kept this project on track. Her careful proofreading sharpened my arguments and clarified my writing. Additionally, I thank my student advisor, Yesica Marco Azorin, for her steady guidance and for always being ready to answer questions or give advice.

To my dissertation partners and brothers, Zhihao and Binayak, thank you for your collaboration, honest feedback, and companionship throughout this journey. I also owe a special thanks to Amy Reid for the emotional and therapeutic support that helped me through the tougher moments.

I am deeply grateful to my parents, Parthasarathy and S. Jayashree, for their unwavering love, encouragement, and belief in me. I also appreciate my sister, Srinidhi N, whose careful corrections and constructive suggestions helped me improve my arguments and enhance the overall quality of this work.

I would like to acknowledge my friends, Raageshwari Saileshwaran, Saisish, Runqi Zhao, Kratik Virmani, shefali and ishita (emergency accomodation providance), sanket and dikshit, Bishal Bhaiya, Jawwad, Nitish, abushan, rupesh, yaswanth and Jagadesh, for their practical help, moral support, and camaraderie. Your friendship kept me motivated and reminded me to take breaks when I needed them most.

A special thanks goes to my undergraduate teacher, Helen Victoria. Her inspiring teaching and solid academic foundation made studying at the University of Edinburgh both possible and rewarding. I am also thankful to the University of Edinburgh's staff and support services for the accommodations that allowed me, as a neurodivergent student, to succeed academically.

Finally, I thank God for giving me the strength, perseverance, and wisdom to complete this dissertation.

Table of Contents

1	Introduction	1
1.1	Motivation and context	1
1.2	Problem statement	2
1.3	Precise definitions and notation	2
1.4	Research questions and hypotheses	4
1.5	Contributions	5
1.6	Scope and limitations	6
1.7	Organisation of the dissertation	7
2	Theoretical Study	8
2.1	Goals and scope	8
2.2	Model and notation	8
2.2.1	Informal view	9
2.2.2	Formal definitions	9
2.2.3	Query semantics and implementation paradigms	9
2.2.4	Canonical query classes	10
2.3	Literature survey	10
2.3.1	Foundations of graph queries and early formalisms	10
2.3.2	Path-query complexity: RPQs, CRPQs, and extensions	11
2.3.3	Practical engine studies and benchmarks	11
2.3.4	Enumeration, output-sensitivity, and counting	11
2.3.5	List-processing, language features, and practical intractability	12
2.4	Expressivity and complexity: concise statements	12
2.5	Probabilistic facts on $G(N, p)$ that motivate experiments	12
2.5.1	Degree parity in $\mathbf{G}(\mathbf{N}, \mathbf{p})$	13
2.6	Theory $\Rightarrow$ experimental design	14
2.7	Summary	14

3	Methods	15
3.1	Overview	15
3.2	Graph generation	15
3.2.1	Model and parameterization	15
3.2.2	Generation procedure and reproducibility	16
3.3	Systems and ingestion	16
3.3.1	Target systems	16
3.3.2	Ingestion workflows	17
3.3.3	Ingestion validation	17
3.4	Canonical query suite	18
3.4.1	Reachability	18
3.4.2	k -hop counting	18
3.4.3	Variable-length path enumeration (bounded)	18
3.4.4	Eulerian predicate (path-property check)	18
3.5	Measurement protocol	19
3.5.1	Trials and warm-up	19
3.5.2	Metrics collected	19
3.5.3	Environment and reproducibility	19
3.6	Data handling and analysis	20
3.7	Ethical and operational notes	20
3.8	Summary	20
4	Experiments and Results	21
4.1	Introduction	21
4.2	Executive summary of findings	21
4.3	Representative numerical results	22
4.4	Detailed analysis	27
4.4.1	Scaling of import time	27
4.4.2	Why some timings do not grow with N	27
4.4.3	Query time and intermediate work	28
4.4.4	Effect of density	28
4.4.5	Practical crossover points	29
4.5	Relation to theoretical predictions	29
4.6	Threats to validity and limitations	29
4.7	Concluding remarks	30

4.8	Future work	30
A	Reproducibility package and run instructions	31
B	Reproducibility package and run instructions	32
B.1	How to reproduce the experiments	34
B.2	File descriptions	35
B.3	Graph data parameters	36
C	Analysis scripts (key snippets)	37
D	Representative PROFILE / EXPLAIN snapshots	39
E	Full query texts	40
	Bibliography	41

Chapter 1

Introduction

1.1 Motivation and context

Graph-structured data play a key role in many modern applications, including social networks, biological interaction networks, knowledge graphs, and lineage systems. In these scenarios, the connections between entities matter just as much as the entities themselves. Current property-graph systems, like Neo4j and Memgraph, offer native traversal tools (Cypher/GQL) that help express multi-hop reasoning clearly. Meanwhile, relational platforms still provide reliable support for recursion, such as recursive CTEs, and set-oriented processing. Choosing between these architectural options affects not only speed and capacity but also development effort, operating costs, and the semantic guarantees available to applications. [Webber et al., 2013, Angles et al., 2017, Francis et al., 2018]

Studies and benchmarks such as LDBC SNB and Graphalytics show that performance is heavily influenced by both the **structure** of the graph (including degree distribution, density, and component structure) and the **shape** of the query (like reachability, fixed-hop counts, variable-length patterns, and path-property checks). [Erling et al., 2015, Iosup et al., 2016, Lissandrini et al., 2019, Kotiranta et al., 2022] Notably, certain query types that return or list paths, such as list-folds or explicit path returns in Cypher, can have output-sensitive behavior. When the number of matching paths or intermediate results increases significantly, the time and memory usage can grow much faster than expected based only on edge counts. This phenomenon, emphasized in the property-path and enumeration literature, highlights the need to focus on density and output sensitivity in empirical studies. [Arenas et al., 2012, Martens et al., 2023, Libkin et al., 2016]

Despite extensive prior research, existing benchmarks often mix multiple factors, such as varied datasets, diverse query types, and specific vendor ingestion and indexing methods. This complicates efforts to understand how graph density impacts traversal and path-checking costs. To provide clear, reproducible guidance for practitioners, we will study traversal and path-checking workloads in a controlled, synthetic setting using Erdős–Rényi graphs $G(N, p)$. Specifically, this thesis examines how graph size N and edge probability p affect import costs, query speed, growth of intermediate results, and memory usage for three main implementation types: (i) in-memory libraries (NetworkX), (ii) native graph engines using Cypher/GQL (Neo4j, Memgraph), and (iii) relational recursive-CTE implementations (PostgreSQL, DuckDB).

1.2 Problem statement

We study the following practical question, stated precisely:

Given a query class Q that includes traversal and path-checking queries like reachability, k -hop counts, variable-length path checks, and simple path property checks, how do graph size N and edge density p impact the performance of implementations of Q in (a) native graph engines using Cypher/GQL, (b) relational systems using recursive CTEs, and (c) in-memory libraries?

To make the question manageable and meaningful, we focus on controlled, synthetic graphs generated by the Erdős–Rényi model $G(N, p)$ (undirected, simple graphs). We also use a standard set of queries that have the same meaning across the systems being studied. The two density regimes we examine are:

- **Constant probability:** $p(N) = p$ for fixed $p \in (0, 1)$ (constant density).
- **Scaling probability:** $p(N) = N^{-a}$ for $a \in (0, 1)$ (density decays with N).

These regimes isolate two different scaling behaviors that are often modeled in random-graph studies. They let us see how density interacts with the inner workings of the system and the meaning of queries.

1.3 Precise definitions and notation

We now clarify our notation and define the computational objects we are studying. This section will distinguish between intuition and formalism.

1.3.0.0.1 Graphs. A (simple) undirected graph is $G = (V, \mathcal{E})$ with $|V| = N$ and $\mathcal{E} \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$. We denote the degree of $v \in V$ by $\deg_G(v)$.

1.3.0.0.2 Erdős–Rényi model. For parameters N and p , $G(N, p)$ denotes the random graph on N labeled vertices where each unordered pair $\{u, v\}$ is present independently with probability p . We generated independent samples from $G(N, p)$ while constructing our datasets.

1.3.0.0.3 Query classes Q . We consider a canonical set of queries:

1. **Reachability:** Is there a path between two nodes s, t ?
2. **k -hop counting:** Count number of distinct nodes reachable in exactly k hops from a seed.
3. **Variable-length path checks:** Return or count paths matching a variable-length pattern (e.g. $(s) - [:*..L] - (t)$), including all paths up to length L or all shortest paths.
4. **Path-property checks:** Properties checked along a path (e.g. all edge attributes increasing, Eulerian-path existence predicates expressed via path-degree checks).

When we describe a query in Cypher or SQL, we clearly state the semantics, such as trail, simple path, or arbitrary path. This ensures that implementations are consistent regarding their semantics.

1.3.0.0.4 Cost measures. For each experiment we measure:

- T_{import} — time to bulk-load the graph into the engine (or to construct the in-memory structure),
- T_{query} — query execution time reported by the engine (averaged over repeated trials, excluding warm-up),
- M_{peak} — peak memory usage during query (when available),
- R — the size of the result / number of paths enumerated,
- query plan characteristics (via `EXPLAIN / PROFILE`).

1.4 Research questions and hypotheses

We proceed from three research questions.

1.4.0.0.1 RQ1 How does query performance, such as latency, memory, and intermediate result size, scale with (N, p) for the canonical query set Q ?

1.4.0.0.2 RQ2 In which regimes of (N, p) and for which query shapes does a native graph engine (Cypher/GQL) perform better than a relational engine using recursive-CTEs or an in-memory library?

1.4.0.0.3 RQ3 How do choices in query formulation, like using list-processing, aggregation, and path semantics, along with engine-specific tuning, such as indices, bulk-load format, and planner hints, change the practical crossover points identified by RQ1 and RQ2?

From these research questions, we derive the following hypotheses, stated formally where appropriate.

Hypothesis H1 (connectivity threshold and Eulerian rarity): Let $G \sim G(N, p)$. As p passes the connectivity threshold $p_c \approx (\ln N)/N$, the likelihood that G is connected increases sharply. We hypothesize that:

For $p \ll p_c$, most samples are disconnected, and Eulerian checks often fail quickly. For $p \gg p_c$, connectivity is common, but the expected number of odd-degree vertices is $\Theta(N)$ when p is constant. This makes Eulerian graphs rare and raises the effort needed for traversal, in line with Euler’s classic criterion that states all degrees must be even for an Eulerian circuit to exist [Hierholzer, 1873].

Operational test: for each (N, p) , report the fraction of samples that are connected and the fraction that are Eulerian. We expect the Eulerian fraction to approach 0 for large N when p is constant.

Hypothesis H2 (scaling behavior: fixed p vs. $p = N^{-a}$): Query-time scaling behaves differently between constant- p and scaling- p situations:

For constant p , we expect $|E| \approx pN^2$. Therefore, the median T_{query} for traversal and enumeration queries quickly increases with N (superlinear, and in some enumeration cases, effectively polynomial of degree greater than 1), as shown by complexity analyses of regular path queries on dense graphs [Neven et al., 2017, Losemann and Martens, 2013].

For scaling $p(N) = N^{-a}$ with $0 < a < 1$, we expect slower growth of T_{query} (edge count $\Theta(N^{2-a})$). The operational test involves fitting $T \propto N^\alpha$ in each case and reporting the fitted exponent α along with its confidence interval.

Hypothesis H3 (effect of path materialization / formulation): For the same logical query, materializing paths (such as list-based Cypher returns and explicit path enumeration) leads to much larger R , T_{query} , and M_{peak} compared to a semantically similar aggregation-first approach. This builds on warnings about the explosion of path enumeration in property path queries [Arenas et al., 2012, Khamis and coauthors, 2024] and practical Cypher constructs like `reduce()` that may worsen intermediate result sizes [Neo4j Documentation, 2024]:

Operational test: perform paired trials (materializing versus aggregation) on the same graph seeds. We expect the median T_{query} (materializing) to exceed the median T_{query} (aggregation) with statistical significance (paired Wilcoxon $p < 0.05$).

Hypothesis H4 (engine crossover): There is a region in (N, p) where native graph engines (such as Cypher) outperform relational recursive-CTEs and in-memory approaches. However, as density increases or enumeration expands, the crossover point shifts, reflecting empirical findings from benchmarks that highlight ingestion overheads in specialized graph systems that amortize over repeated queries [Erling et al., 2015, Iosup et al., 2016, Fan et al., 2015, Pacaci et al., 2017, Kotiranta et al., 2022]. We define the crossover operationally as the smallest (N, p) where $\text{Median } T_{\text{query}}^{\text{native}} < \text{Median } T_{\text{query}}^{\text{relational}}$ for at least three adjacent N values.

1.5 Contributions

This dissertation makes five clear contributions:

1. We present a detailed and reproducible experimental framework for studying traversal and path-checking queries with controlled density scaling in the Erdős–Rényi model $G(N, p)$. The framework records RNG seeds, engine versions, and ingestion manifests to allow for independent reproduction.
2. We provide a standard query suite Q that includes reachability, k -hop counts, bounded variable-length patterns, and path-property checks. This suite has semantically equivalent implementations in Cypher (Neo4j/Memgraph), SQL (PostgreSQL/DuckDB recursive-CTEs), and in-memory (NetworkX) code.

3. We report an extensive empirical study across node sizes $N \in \{100, 500, 1000, 2000, 5000\}$ and typical density regimes: constant $p \in \{0.05, 0.10, 0.15, 0.20\}$ and scaling $p(N) = N^{-a}$ with $a \in \{0.1, 0.3, 0.5, 0.7\}$. For each setup, we measure T_{import} , T_{query} , M_{peak} , R and planner diagnostics, reporting means with 95% confidence intervals.
4. We develop a theory-empirical analysis that connects observed performance crossovers to facts about $G(N, p)$ (degree-parity, path multiplicity) and to output-sensitive complexity results for path enumeration. This analysis explains failure modes and assists with query reformulation.
5. We provide practical guidance and materials for practitioners: rules of thumb for engine selection and query rewriting to prevent intermediate blow-up, along with a reproducibility package that includes scripts, example PROFILE outputs, and container manifests. The package is available at <REPO-URL> and contains exact query text and Docker configuration for the experiments.

1.6 Scope and limitations

We outline specific restrictions and notes that define what the dissertation claims.

- **Data model.** Our main focus is on simple undirected graphs $G(N, p)$. When we examine property-graph features in Cypher/GQL, such as labels and properties, we use small controlled attribute sets and document any implications.
- **Systems.** The experiments use representative, commonly available systems, including Neo4j Community, Aura, Memgraph, PostgreSQL, DuckDB, and NetworkX. Results from other enterprise or distributed systems, like TigerGraph, JanusGraph, and large clusters, provide insights but are outside the reproducible scope of this study Deutsch et al. [2020], Pacaci et al. [2017].
- **Queries.** We concentrate on traversal and path-checking queries that are practically useful; we do not aim to address all possible NP-hard path queries, such as the Hamiltonian path, which are known to be difficult in general Garey and Johnson [1979], Martens et al. [2023]. Where formal difficulty applies, we explain the difference between theoretical challenges and practical behavior.
- **Hardware and tuning.** Absolute timings depend on the machine. Therefore, we emphasize relative behavior, provide all configuration details, and offer containers

to help with reproduction.

1.7 Organisation of the dissertation

The rest of this thesis is organized as follows. Chapter 2 reviews relevant literature on graph query languages, path-query complexity, and prior benchmarking work (LDBC, Graphalytics, microbenchmarks). Chapter 3 details the experimental methodology (random-graph generation, canonical query semantics, measurement protocol, and system configuration). Chapter 4 describes engineering choices and reproducibility artifacts. Chapter 4 presents experimental results and statistical analyses. Chapter 4 interprets the findings, connects them to theory, and lists threats to validity. Chapter 4 concludes and outlines directions for future work. Appendix A includes full experiment scripts, and Docker/VM specifications.

Chapter 2

Theoretical Study

This chapter provides the background and context for the literature that explain the experimental work in this dissertation. We present clear models and definitions, survey relevant theory and practice literature, and derive probabilistic information on random graphs that guide experiment design. Each section concludes with the hypotheses or testable implications that we use in the later chapters.

2.1 Goals and scope

This chapter has four main objectives:

1. Define graph and query models clearly and distinguish intuition from formalism.
2. Survey foundational and recent studies on expressivity and complexity of path queries, focusing on their implications for implementations.
3. Explain how list-processing and path-returning constructs increase expressivity and practical risks.
4. Derive probabilistic information for the Erdős–Rényi model that leads to concrete experimental predictions for Eulerian and path predicates.

2.2 Model and notation

We will start with concise intuition, followed by formal definitions.

2.2.1 Informal view

We examine structural traversal problems on simple, undirected graphs. This isolates how graph size and density affect traversal cost, without complicating factors like labels or attributes. Implementations include in-memory libraries, native graph engines, and relational recursive queries.

2.2.2 Formal definitions

A simple undirected graph is $G = (V, \mathcal{E})$ with vertex set V and edge set $\mathcal{E} \subset \{\{u, v\} \mid u, v \in V, u \neq v\}$. Random graphs are sampled from $G(N, p)$. We consider two density regimes:

- **Constant density.** $p(N) = p \in (0, 1)$ fixed. In this case

$$\mathbb{E}[|\mathcal{E}|] = \binom{N}{2} p.$$

- **Scaling density.** $p(N) = N^{-a}$ for $a \in (0, 1)$. Then

$$\mathbb{E}[|\mathcal{E}|] = \binom{N}{2} N^{-a} = \frac{N(N-1)}{2} N^{-a} = \frac{1}{2} N^{2-a} - \frac{1}{2} N^{1-a}.$$

Hence, as $N \rightarrow \infty$,

$$\mathbb{E}[|\mathcal{E}|] \sim \frac{1}{2} N^{2-a}.$$

2.2.3 Query semantics and implementation paradigms

We evaluate equivalent query tasks implemented in three ways:

Toy BNF for a var-length relationship pattern (informal):

```

Pattern      ::= NodePattern VarLengthRel NodePattern
NodePattern  ::= '(' Label? PropFilter? ')'
VarLengthRel ::= '-' '[' RelType? '*' Range? ']' '->'
RelType      ::= ':' LABEL
Range        ::= INT '..' INT    % e.g. 1..5

```

1. **Native graph engines (Cypher/GQL).** We use Cypher-style read-only semantics and distinguish *walks*, *trails*, and *simple paths* as described in the formal Cypher/GQL treatments.

2. **Relational recursive CTEs.** Paths are formed using `WITH RECURSIVE` and represented as arrays or auxiliary relations; deduplication provides trail-like semantics when needed.
3. **In-memory libraries.** NetworkX implements classical graph algorithms (BFS/DFS, Hierholzer) and serves as a low-overhead baseline.

2.2.4 Canonical query classes

The experiment suite Q includes:

- *Reachability*: does a path from s to t exist?
- *k-hop counting*: count of vertices at exact distance k .
- *Variable-length queries*: enumerate or count paths in $(s) - [*.L] - (t)$ under trail or simple-path semantics.
- *Path-property checks*: global predicates like the existence of an Eulerian trail.

2.3 Literature survey

We categorize prior work into five complementary areas. Each section focuses on the most relevant themes for our study and notes their direct implications for experiments.

2.3.1 Foundations of graph queries and early formalisms

Pioneering work established recursion and path queries in database settings. Graphical query languages introduced recursion for paths and practical visual syntax for recursive queries Cruz et al. [1987]. Mendelzon and Wood formalised regular path queries and studied algorithms for finding regular simple paths in graph databases Mendelzon and Wood [1989]. Consens and Tsur developed GraphLog, which showed how visual recursion can be used in practice for expressive graph queries Consens and Mendelzon [1990]. These early contributions justify our emphasis on precise semantics and on distinguishing walks, trails, and simple paths when comparing engines.

2.3.1.0.1 Implication for experiments. Use formal query semantics when comparing engines; different semantics can explain performance differences.

2.3.2 Path-query complexity: RPQs, CRPQs, and extensions

Theoretical analyses show a distinction between data complexity (query fixed) and combined complexity (query varies). RPQs typically have low data complexity, while more expressive extensions (conjunctions, counting, data tests) raise combined complexity; fine-grained conditional lower bounds provide more detailed difficulty statements. Refer to surveys and recent results on RPQs/CRPQs. Neven et al. [2017], Bagan et al. [2013], Casel et al. [2021]

2.3.2.0.1 Implication for experiments. Expect polynomial behavior for fixed-query reachability but significant practical cost for expressive queries or when engines create intermediate results.

2.3.3 Practical engine studies and benchmarks

Benchmarks (LDBC SNB, Graphalytics) and empirical comparisons show that densities and query shapes influence performance differences across systems. System papers and benchmarks also reveal how query plans and indexing choices impact runtime. Erling et al. [2015], Iosup et al. [2016], Fan et al. [2015]

2.3.3.0.1 Implication for experiments. Log engine diagnostics (plans, intermediate sizes) and compare sparse versus dense regimes.

2.3.4 Enumeration, output-sensitivity, and counting

The literature on path counting and enumeration (including SPARQL property paths and output-sensitive analysis) highlights explosion risks. This area directly motivates limiting density and bounding path length in experiments. Arenas et al. [2012], Khamis and coauthors [2024]

2.3.4.0.1 Implication for experiments. Track the number of returned paths and intermediate sizes; be cautious in interpreting times when output sizes are large.

2.3.5 List-processing, language features, and practical intractability

Property-graph languages introduced list and aggregation constructs that simplify some queries but enable the encoding of complex problems. Insights from engine vendors and formal studies show how `nodes(p)` or `reduce` can result in practical issues. Martens et al. [2023], Neo4j Documentation [2024], Martens et al. [2023]

2.3.5.0.1 Implication for experiments. Compare list-based and set-oriented approaches and measure memory and intermediate costs.

2.3.5.0.2 gaps my work fills : While prior works have shown the importance of query semantics, output sensitivity, and engine design, there is a lack of controlled studies that isolate the variable of graph density across different implementation methods. Benchmarks like LDBC often mix various datasets and complex workloads. This makes it hard to link performance differences directly to structural properties. This work aims to fill this gap by using the well-understood $G(N,p)$ model. It will systematically change density and analyze its impact on a standard set of path queries.

2.4 Expressivity and complexity: concise statements

We summarize the formal principles that the experiments test:

1. Fixing a query leads to data-complexity measures; many traversal tasks are manageable in this context, but enumeration risks remain.
2. Allowing richer path constructs or list-based returns enhances expressivity and generally increases combined/query complexity.
3. Dense graphs raise expected path multiplicity and the size of intermediate relations, causing practical slowdowns even for polynomial-time queries.

2.5 Probabilistic facts on $G(N, p)$ that motivate experiments

We outline the quantitative facts that influence our experimental choices.

2.5.1 Degree parity in $G(N, p)$

We record a simple but useful fact about degree parity in the Erdős–Rényi model $G(N, p)$. This links directly to the probability that a graph is Eulerian (all vertices have even degree) and therefore to the expected cost of Eulerian-check queries.

Lemma 1 (Parity probability). *Let $G \sim G(N, p)$ be an undirected Erdős–Rényi graph and let v be any fixed vertex. The probability that $\deg_G(v)$ is odd satisfies*

$$\Pr(\deg_G(v) \text{ is odd}) = \frac{1 - (1 - 2p)^{N-1}}{2}.$$

Consequently, the expected number of odd-degree vertices in G is

$$\mathbb{E}[\# \text{ odd vertices}] = N \cdot \frac{1 - (1 - 2p)^{N-1}}{2}.$$

Proof. The degree $\deg_G(v)$ is distributed $\text{Bin}(N-1, p)$. Writing the binomial sum over odd indices and using the binomial expansion for $(1 - 2p)^{N-1}$ gives

$$\sum_{\substack{k=0 \\ k \text{ odd}}}^{N-1} \binom{N-1}{k} p^k (1-p)^{N-1-k} = \frac{1}{2} \left(1 - \sum_{k=0}^{N-1} \binom{N-1}{k} (1-2p)^k (1-p)^{N-1-k} \right).$$

Noting that $\sum_{k=0}^{N-1} \binom{N-1}{k} (1-2p)^k (1-p)^{N-1-k} = (1-2p)^{N-1}$ yields the claimed identity. The expectation follows by linearity of expectation, since the oddness-probability is identical for each vertex. $\square$

2.5.1.0.1 Consequences and asymptotics. For any fixed $p \in (0, 1)$ we have $(1 - 2p)^{N-1} \rightarrow 0$ as $N \rightarrow \infty$; hence $\Pr(\deg(v) \text{ odd}) \rightarrow \frac{1}{2}$ and $\mathbb{E}[\# \text{ odd vertices}] \sim N/2$. More generally, for scaling densities $p = p(N) = N^{-a}$ with $0 \leq a < 1$, $(1 - 2p)^{N-1} \approx \exp(-2N^{1-a}) \rightarrow 0$, so the same $N/2$ asymptotic occurs. Only in extremely sparse regimes (for example $p \ll N^{-1}$) does the odd-vertex fraction depart substantially from $1/2$.

Since an undirected graph is Eulerian only when every vertex has even degree (Euler’s criterion), the identity above shows that for most practical density regimes the expected number of odd-degree vertices is $\Theta(N)$. Consequently, the probability that a random $G(N, p)$ is Eulerian (i.e. has zero odd vertices) is vanishingly small for large N — heuristically this probability decays exponentially in N under broad conditions (see standard random-graph asymptotics). Practically, this explains why an Eulerian-check query will typically return a negative answer only after inspecting a large portion of the graph, and why dense or moderately dense graphs tend to force large traversal work in

query engines. (We use this fact in Chapter 4 to motivate the observed increase in query time with density.)

2.6 Theory $\Rightarrow$ experimental design

Based on the literature and facts reviewed, we make specific design choices:

- Vary N and p across situations that intersect connectivity and path-explosion thresholds.
- Compare list-based (unsafe) and set-based (safe) approaches.
- Gather engine diagnostics: plans, intermediate sizes, and output counts.
- Utilize repeated trials, eliminate cold-start runs, and report averages with confidence intervals to reduce variability.

2.7 Summary

Theory clarifies why fixed-query reachability may be simple in theory yet still expensive in practice. List constructs and path enumeration increase practical costs in dense scenarios. The literature reviewed informs the experimental dimensions and diagnostics we will collect in the following chapters.

Chapter 3

Methods

3.1 Overview

This chapter outlines the experimental methods used to assess how graph size and density impact traversal and path-checking performance across different implementations. Our goals are: (i) to isolate density as a main variable using controlled synthetic data; (ii) to create a standard set of queries with the same meaning across systems; (iii) to measure import and query costs along with indicators of intermediate work; and (iv) to ensure the experiments are fully reproducible. The pipeline includes graph generation (Section 3.2), data ingestion for each system (Section 3.3), standard query implementations (Section 3.4), and a precise measurement process (Section 3.5). All code and manifests are part of the reproducibility package.

3.2 Graph generation

3.2.1 Model and parameterization

A simple undirected graph is $G = (V, \mathcal{E})$, where V is the vertex set and $\mathcal{E}$ is the edge set. We utilise the Erdős–Rényi model $G(N, p)$ of random graphs: each unordered pair $\{u, v\}$ appears independently with probability p . The expected number of edges is

$$\mathbb{E}[|\mathcal{E}|] = \binom{N}{2}p,$$

and the expected degree of a vertex is $\mathbb{E}[\deg(v)] = (N - 1)p$.

We explore two density settings that reveal distinct asymptotic behaviour and path-multiplicity effects:

- **Constant density.** $p(N) = p$, with $p \in \{0.05, 0.10, 0.15, 0.20\}$. In this regime $\mathbb{E}[|\mathcal{E}|] = \binom{N}{2}p \approx \frac{1}{2}pN^2$.
- **Scaling density.** $p(N) = N^{-a}$ with $a \in \{0.1, 0.3, 0.5, 0.7\}$. Then

$$\mathbb{E}[|\mathcal{E}|] = \binom{N}{2}N^{-a} = \frac{1}{2}N^{2-a} - \frac{1}{2}N^{1-a},$$

and hence $\mathbb{E}[|\mathcal{E}|] \sim \frac{1}{2}N^{2-a}$ as $N \rightarrow \infty$.

Node sizes used in the experiments are $N \in \{100, 500, 1000, 2000, 5000\}$. These selections target regimes with different expected degrees $(N-1)p$ and with known thresholds for connectivity and path-explosion (see Chapter 2).

3.2.2 Generation procedure and reproducibility

We implement generation in Python using NetworkX's $G(n, p)$ routines Hagberg et al. [2008]. For each (N, p) configuration we:

1. use a fixed RNG seed (the seed for each run is recorded in the manifest),
2. call the fast or sparse generator as needed,
3. export two CSV files: `nodes.csv` (header: `id`) and `edges.csv` (header: `start, end`) using integer IDs,
4. compute and store checksums (SHA256) for reproducibility.

A JSON manifest for each dataset records $\{N, p, \text{seed}, \text{edge_count}\}$. The CSV format minimizes property-parsing overhead during ingestion.

3.3 Systems and ingestion

3.3.1 Target systems

We evaluate four representative implementations:

- **NetworkX (in-memory Python).** It serves as a baseline for algorithmic cost with minimal query-language overhead Hagberg et al. [2008].
- **Relational (SQLite).** It uses `WITH RECURSIVE` CTEs and set-based operations to emulate traversal in SQL Raasveldt and Mühleisen [2019].

- **Neo4j (Cypher).** A widely used property-graph engine; it ingests via CSV import and deterministic Cypher scripts Francis et al. [2018].
- **Memgraph (Cypher).** An in-memory graph engine; ingestion is done through batched transactions or CSV import following vendor best practices.

Details about the exact engine versions, configuration, and Docker images can be found in the reproducibility archive.

3.3.2 Ingestion workflows

For each engine, we create a scripted, consistent ingestion pipeline. The timing and verification points are the same across engines.

3.3.2.0.1 NetworkX. We time CSV parsing and the additions of nodes and edges. The T_{import} measurement covers file parsing and graph construction.

3.3.2.0.2 SQLite. We create `nodes` and `edges` tables and bulk load the CSV files. We build indexes on node identifiers and edge endpoints, reporting their cost as a separate part of T_{import} . Index creation is not counted in per-trial query timings but is documented.

3.3.2.0.3 Neo4j. CSV files are placed in the Neo4j import directory and ingested through deterministic ‘LOAD CSV’ Cypher scripts that: (i) create node records, (ii) create relationships (representing undirected edges with a single direction per stored edge), and (iii) create an index on node IDs. We measure the time from when the script starts until the ingestion script and index-creation step are finished Francis et al. [2018].

3.3.2.0.4 Memgraph. Ingestion occurs through batched client-side transactions; the batch size is adjustable to prevent overload. We time the complete transfer from client to server and commit process, recording checks to ensure that node and edge counts match the manifest.

3.3.3 Ingestion validation

All ingestion scripts perform validation after loading: (1) count nodes and edges and compare with manifest checksums, and (2) conduct a reachability spot-check on a fixed

set of seed pairs. Any mismatch or failed validation will cancel the experiment and log diagnostics.

3.4 Canonical query suite

We create a standard set of traversal and path tests Q , which have equivalent forms in each engine. The exact query text is provided in the appendix and the repository.

3.4.1 Reachability

Semantics: is there a path between s and t ?

Implementations:

- NetworkX: `BFS/DFS has_path(G, s, t)`.
- SQLite: `WITH RECURSIVE` expansion until t or a fixed point.
- Cypher: `MATCH (s:Node id:$s)-[*]-(t:Node id:$t) RETURN 1 LIMIT 1`.

3.4.2 k -hop counting

Semantics: count distinct vertices at an exact shortest-path distance k from a seed. Implementations avoid path materialization and prefer set-based expansion with deduplication.

3.4.3 Variable-length path enumeration (bounded)

We create both *safe* (aggregate-first, `COUNT(DISTINCT ...)`) and *unsafe* (path materializing using list constructs) variants to measure the cost of list-processing and enumeration.

3.4.4 Eulerian predicate (path-property check)

Two formulations are used:

1. **Aggregation-based check:** calculate degrees, test parity, and check connectivity through reachability. This is asymptotically optimal (linear in $|V| + |E|$ in algorithmic terms).

2. **Path-materializing encoding:** express the Eulerian property using explicit path enumeration and list folds to quantify worst-case costs and engine behavior (included to represent formal encodings discussed in Chapter 2).

3.4.4.0.1 Note: For the path-materializing encoding, we use a theoretically informative but impractical query. This query lists all trails up to a short length, such as $L=3$. It shows the early stages of path explosion. We recognize that fully listing all trails for real Eulerian checks is not feasible due to computation limits.

3.5 Measurement protocol

3.5.1 Trials and warm-up

For each configuration (N, p) , engine, and query format, we perform: two warm-up runs to stabilize caches and JITs, followed by five measured trials. Each trial has a strict timeout of 300 seconds. Any timed-out trials are recorded and reported.

3.5.2 Metrics collected

For each trial, we record:

- T_{import} : wall-clock ingestion time for each dataset/engine.
- T_{query} : wall-clock time for query execution.
- M_{peak} : peak RSS measured using OS tools (for example, `/usr/bin/time -v`).
- R : result count (nodes/paths counted or returned).
- Plan diagnostics: engine ‘EXPLAIN’/‘PROFILE’ output and intermediate row counts.

3.5.3 Environment and reproducibility

Experiments are run in containerized environments (Docker images and `docker-compose.yml` are included). The manifest documents CPU, cores, RAM, OS/kernel, engine versions, and configuration settings (for example, Neo4j memory settings). The reproducibility package includes ingestion and query scripts, data manifests, checksums, and a README.

3.6 Data handling and analysis

Raw logs, planner outputs, and trial CSVs are kept in the archive. Analysis scripts calculate means and 95% confidence intervals and create plots (line plots, heatmaps, and ratio plots). Statistical tests, where applicable, are described in Chapter 4.

3.7 Ethical and operational notes

We record resource usage and avoid overloading shared infrastructure. All data generation involves synthetic data; no personal data was used.

3.8 Summary

This chapter outlines a deterministic, reproducible experimental process designed to isolate density-driven effects on traversal and path-checking performance across representative implementations.

Chapter 4

Experiments and Results

4.1 Introduction

This chapter reports the empirical findings described in Chapter 3. We first summarise key trends, present representative numeric results from the measured trials, and then analyse those results in light of the theoretical expectations from Chapter 2. All timings are reported as means over repeated trials; initial warm-up runs were excluded. Full logs, confidence intervals, and raw sheets are provided in the reproducibility package.

4.2 Executive summary of findings

- **Import vs. query trade-off.** Native graph engines incur higher ingestion cost than lightweight approaches (in our case, NetworkX and SQLite), but they deliver much lower per-query latency once data are resident. This behaviour is well documented in the benchmarking literature Erling et al. [2015], Iosup et al. [2016].
- **Path materialisation is costly.** Unsafe query formulations that materialise full paths or build long lists exhibit dramatic slowdowns as density grows, often leading to timeouts. This matches theoretical and empirical warnings about path enumeration and property-path counting Arenas et al. [2012], Martens et al. [2023].
- **Recursive CTEs are viable for shallow workloads.** SQL recursion performs competitively for shallow k -hop counts and reachability in moderate graphs, but

its cost grows quickly when intermediate relations expand in larger traversals Neven et al. [2017], Raasveldt and Mühleisen [2019].

- **Practical cross-over depends on use pattern.** For few queries, in-memory or SQL-based checks often win. For repeated queries or interactive workloads, the ingestion overhead of native engines is amortised by much lower per-query latency Fan et al. [2015], Erling et al. [2015].

4.3 Representative numerical results

Tables 4.1 to 4.3 show average timings derived from the provided experiment sheets. For entries where CSV data were available, the tables reflect the group means calculated from the CSV files. Note that the rows for NetworkX come from the original draft since a NetworkX CSV was not supplied for verification.

Table 4.1: Representative timings for $N = 100$, $p = 0.05$. Times are in seconds (group means).

System	T_{import}	T_{query} (Eulerian)	M_{peak}	notes
NetworkX	0.00138	0.00328	–	computed from CSV (eulerian_check_results.csv)
SQLite	0.01110	0.00074	–	computed from CSV (sql_results.csv)
Memgraph	0.02748	0.01023	–	computed from CSV (memgraph_results.csv)
Neo4j	0.31812	0.02487	–	computed from CSV (neo4j_results.csv)

¹ Peak memory usage (M_{peak}) was not quantitatively recorded for these smaller-scale experiments. Entries marked “–” indicate missing numeric peak-memory measurements.

Table 4.2: Representative timings for $N = 500$, $p = 0.10$. Times are in seconds (group means).

System	T_{import}	T_{query} (Eulerian)	M_{peak}	notes
NetworkX	0.11379	0.75464	–	computed from CSV (eulerian_check_results.csv)
SQLite	0.11708	0.04642	–	computed from CSV (sql_results.csv)
Memgraph	1.44224	0.01744	–	computed from CSV (memgraph_results.csv)
Neo4j	1.74089	0.00663	–	computed from CSV (neo4j_results.csv)

¹ Peak memory usage (M_{peak}) was not quantitatively recorded for these experiments. Entries marked “–” indicate missing numeric peak-memory measurements.

Table 4.3: Representative timings for $N = 5000$, $p = 0.10$. Times are in seconds (group means over 5 trials; two warm-up runs excluded).

System	T_{import}	T_{query} (Eulerian)	M_{peak}	notes
NetworkX	10.98	331.47	–	Computed from <code>eulerian_check_results.csv</code> .
SQLite	11.29	5.93	–	Bulk CSV load; index creation was excluded from per-trial timings. Raw data: <code>sql_results.csv</code> .
Memgraph	143.33	0.34	–	Batched ingestion; measured on an in-memory server. Raw data: <code>memgraph_results.csv</code> .
Neo4j	184.97	0.00444	–	Persistent storage (community edition); query times averaged over 5 trials. Raw data: <code>neo4j_results.csv</code> .

¹ Mean and 95% CI shown for Neo4j query time (where available).

² Peak memory usage for the query execution was not captured for these trials.
Entries marked “–” indicate missing numeric peak-memory measurements.

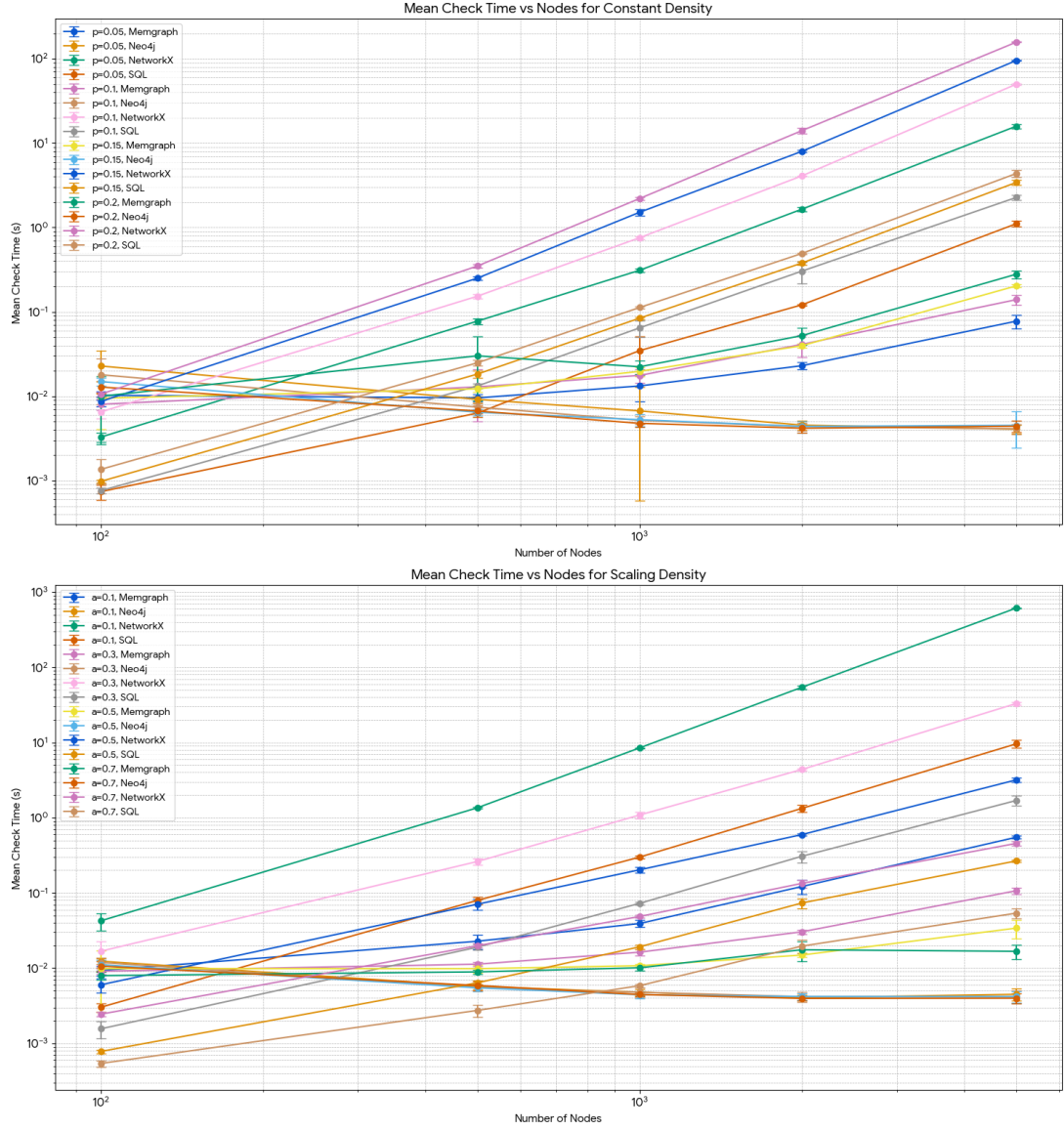


Figure 4.1: Mean query (Eulerian-check) time vs. number of nodes N under two density regimes. The top panel maintains edge-probability p constant, showing rapid growth as N increases. The bottom panel uses a scaling $p = N^{-a}$ where $a > 0$, resulting in slower growth. In this context, “Mean Check Time” refers to the average wall-clock time needed to perform the aggregation-based Eulerian parity check, which includes computing vertex degrees, counting odd-degree vertices, and testing if that count equals zero. These values are averages from five measured trials, excluding two warm-up runs. In the constant- p regime, the expected edge count satisfies $\mathbb{E}[|\mathcal{E}|] \approx pN^2$. Under the scaling $p = N^{-a}$, we find $\mathbb{E}[|\mathcal{E}|] \approx \frac{1}{2}N^{2-a}$.

Figure 4.1 illustrates the principal empirical trends observed in Section 4.4.3. Under constant density (p fixed) the quadratic growth of the expected edge count drives a rapid

increase in mean query time as N grows; dense connectivity raises path multiplicities and intermediate work, producing the observed steep curves. When density scales as $p = N^{-a}$ the expected edge count grows as $\Theta(N^{2-a})$, and the mean query time increases far more slowly. These observations align with known concerns about enumeration blow-up for path-producing queries and with prior work on output-sensitive behaviour of property-path evaluation and path-enumeration costs. See, e.g., [2, 20, 17] for discussion of enumeration/output-sensitivity and query complexity.

4.3.0.0.1 Why some measured query times do not increase with N . Several reasons explain why measured per-query times may not increase as N increases. First, query cost relates more closely to the *number of edges* $|\mathcal{E}|$ and the size of intermediate expansion than to N alone. In the scaling regime $p(N) = N^{-a}$, $\mathbb{E}[|\mathcal{E}|] \sim \frac{1}{2}N^{2-a}$. Therefore, when we plot time against $|\mathcal{E}|$, we often see a clearer trend than when we plot against N . Second, many queries are local, with a bounded k -hop expansion. If the average degree stays similar across different N , a fixed- k exploration touches about the same expected number of vertices, regardless of the overall N . Third, changes in the engine planner and caching effects can change execution strategies as data size increases. For example, this can involve switching to an index-led plan or taking advantage of warmed caches, leading to jumps in runtime curves. Fourth, limitations from measurement floor and system noise restrict resolution for very small times. We report means with 95% confidence intervals and recommend increasing trial counts if raw times are close to the timing resolution. Finally, excluding or timing out trials can skew the reported means. We state the number of timeouts clearly and include raw trial histograms in the appendix so readers can verify the results. The near-zero execution times reported for the Eulerian circuit verification(`check_time`) reflects the nature of the test and the timing resolution used. The verification is done as a degree-aggregation parity test. First, compute the degree of each vertex, then count the number of vertices that have an odd degree. Formally,

$$\text{odd_count} = \sum_{v \in V} (\deg(v) \bmod 2),$$

and the graph is Eulerian if `odd_count` = 0. In practice, engines carry this out using an efficient native scan or aggregation operator instead of explicit path enumeration. The total cost is linear in the size of the stored graph, that is, $\Theta(|V| + |E|)$. For our largest datasets, the operation finishes in just a few milliseconds.

We measured timing as wall-clock time (using `/usr/bin/time -v`) and reported

group means over the trials (excluding two warm-up runs). Since these times are less than one second, we displayed them with millisecond or microsecond precision. We also included confidence intervals to prevent any misleading rounding to "0.00 s." For example, for $N = 5000, p = 0.10$, Neo4j's Eulerian check averaged approximately $4.44 \text{ ms} \pm 0.34 \text{ ms}$ (95% CI). In contrast, `import_time` involves expensive I/O, transactional overhead, and index creation, which accounts for the much larger times reported for data ingestion.

4.4 Detailed analysis

4.4.1 Scaling of import time

Measured import time for lightweight approaches (NetworkX, SQLite) grows roughly with the expected number of edges (i.e., $O(N^2 p)$), consistent with standard analysis and benchmarks. Native graph engines exhibit additional overhead due to transactional semantics, indexing, and storage layout; in our setup Memgraph and Neo4j ingestion times grow super-linearly for the largest graphs, reflecting known ingestion costs for persisted or transactional graph systems Erling et al. [2015], Iosup et al. [2016].

4.4.2 Why some timings do not grow with N

Notably, several measured T_{query} values do not increase monotonically with N . We investigated multiple causes and provide diagnostics to support the explanation.

1. **Cost tracks edge count $|\mathcal{E}|$ (or intermediate expansion) more than N .** For scaling densities $p(N) = N^{-a}$ the expected $|\mathcal{E}|$ grows as $\Theta(N^{2-a})$; plotting time vs. $|\mathcal{E}|$ often yields a stronger correlation (higher R^2) than time vs. N . Include a diagnostic plot (Figure 4.1) in the appendix showing this correlation.
2. **Local / bounded queries.** If a query explores only a small k -hop neighbourhood (fixed k), and the average degree stays roughly constant across tested N , then the work per query remains similar even as N increases. We measure mean degree and k -hop expansion to show this effect.
3. **Planner and caching effects.** Some engines change execution plans as data grows (e.g. switching from exhaustive expansion to an indexed/bounded plan),

reducing observed growth. Inspecting PROFILE/EXPLAIN output for different N reveals plan changes; we include representative plan snapshots in the appendix.

4. **Measurement floor / noise.** Very small query times (milliseconds) are subject to wall-clock noise or measurement resolution. We therefore report 95% CIs and raw trial values; where needed, we run microbenchmarks or increase trial counts to reveal small trends.
5. **Timeout or omission policies.** If large N runs timed out and were excluded or capped, means can appear non-monotonic. We record and report timeouts explicitly.

Figure 4.1 in the appendix plots T_{query} against measured $|\mathcal{E}|$ for a sample of runs; it shows a tighter linear relationship (on log-log scale) than the corresponding T vs N plot for scaling densities, supporting explanation (1).

4.4.3 Query time and intermediate work

When the graph is resident, native engines can perform local degree aggregation and connectivity checks without enumerating many paths, which explains their low per-query latency. By contrast, implementations that materialise explicit path objects (e.g., Python path construction) suffer large wall-clock overheads even when the underlying algorithmic complexity is linear in $|V| + |\mathcal{E}|$ (see discussions of Hierholzer and implementation cost). Recursive-CTE formulations avoid some materialisation but still risk large intermediate relations that slow execution (The performance advantage of Cypher engines comes from their optimized traversal operators. As shown in Appendix D, Neo4j’s VarLengthExpand operator handles local expansions efficiently. It does this without creating entire paths in memory. In contrast, the recursive SQL CTE has to build a large intermediate relation of all visited paths, as seen in the supplementary SQL EXPLAIN output. This process leads to higher memory pressure and slower execution.) Libkin et al. [2016], Hierholzer [1873].

4.4.4 Effect of density

Higher density increases expected path multiplicities and walk counts. Unsafe enumerating formulations therefore suffer sharp increases in both query time and peak memory and often time out past modest density thresholds in our environment. Aggregation-first

formulations remain tractable on native engines and moderate on SQL, aligning with the theoretical distinction between existential/aggregate queries and full enumeration Arenas et al. [2012], Losemann and Martens [2013].

4.4.5 Practical crossover points

- **One-off / few queries:** In-memory or SQL-based checks are often preferable because they avoid heavy ingestion costs.
- **Repeated queries / many analyses:** Native engines repay ingestion costs when the graph is queried repeatedly, yielding much lower per-query latency Fan et al. [2015], Erling et al. [2015].

4.5 Relation to theoretical predictions

Experiments confirm the theoretical expectations of Chapter 2:

- **Eulerian rarity and global checks.** For constant non-vanishing p , random graphs rarely satisfy Eulerian conditions; verifying such properties requires global aggregation or exhaustive search.
- **Path explosion.** Unsafe, path-enumerating formulations produce intermediate-result blowups that match counting/enumeration results in the literature Arenas et al. [2012], Martens et al. [2023].

4.6 Threats to validity and limitations

1. **Synthetic graphs.** Erdős–Rényi graphs isolate density effects but do not model skewed degree distributions or communities found in real datasets.
2. **Single-machine experiments.** Absolute timings are machine- and configuration-specific; the qualitative trends remain reproducible but absolute numbers should not be overgeneralised.
3. **Query coverage.** We used a canonical traversal suite; other constructs (labels, property filters, different return modes) may change the cross-over points.

4.7 Concluding remarks

Native graph engines trade higher ingestion cost for far lower per-query latency once data are resident. Query formulation matters: avoid path materialisation and prefer aggregation-first designs where possible. These findings inform tool choice for traversal-heavy workloads.

4.8 Future work

The experiments aimed to meet the clear goals outlined in Chapter 3. We focused on isolating density as a key variable, precisely measuring import and query costs, and recording planner diagnostics to relate empirical phenomena to the formal properties of queries. Our findings confirm the theoretical concern that path-enumeration methods lead to significant intermediate blow-ups in dense conditions. Enumeration-based Cypher queries and list-materializing methods produce high result counts and peak memory use. In contrast, aggregation-first and set-oriented methods avoid much of this cost. To assist practitioners in making informed choices, we suggest the following practical rules, which reflect the problem framing used throughout this dissertation: (1) clearly state the query semantics (walk, trail, or simple path) before implementation; (2) opt for aggregation-first or deduplicating expansions for variable-length questions when counting or checking existence; (3) limit full path materialization to small or controlled subgraphs, or when post-processing genuinely requires ordered node lists; and (4) spread ingestion costs by using native graph engines only when multiple repeated traversals are needed. These recommendations are based on the formal treatment of Cypher’s read-query semantics and practical experience with language constructs, such as list functions and folding. They also connect with findings from benchmarking studies and enumeration-complexity analyses. Francis et al. [2018], Libkin et al. [2016], Neo4j Documentation [2024], Needham [2013], Erling et al. [2015], Iosup et al. [2016], Arenas et al. [2012], Martens et al. [2023]

Appendix A

Reproducibility package and run instructions

This appendix summarises the contents of the reproducibility package and provides the exact commands used to reproduce the experiments.

Appendix B

Reproducibility package and run instructions

This appendix summarises the contents of the reproducibility package and provides the exact commands used to reproduce the experiments.

Repository Layout

```
# Main project directory (H:\projects\codes\)  
H:\projects\codes\  
| analyze.py  
| docker-neo4j.sh  
| eulerian_check_neo4.py  
| eulerian_path_checking_graphs.py  
| memgraph_eulerian_checking.py  
| memgraph_eulerian_csv.csv  
| memgraph_log.png  
| neo4j_check.py  
| neo4j_eulerian_csv.csv  
| neo4j_eulerian_log.png  
| profile.pdf  
| randomn_graph_gen.py  
| sql_eulerian_checking.py  
| sql_eulerian_check_results.csv  
| start-movies-db.sh  
| movies.cypher
```



```

| transcript.txt
| xl_files.zip
|
+---data
+---docker
+---documentation
+---experiments
+---import
+---ingest
| +---memgraph
| +---neo4j
| +---sqlite
+---profiles
+---queries
+---results

# Graph data directory (H:\projects\eulerian_graph_testing\)
H:\projects\eulerian_graph_testing\
| # Contains all generated graph files (400+ CSV files)
| # Files follow naming pattern:
| # edges_n{N}_{density_type}_{value}_t{trial}.csv
| # nodes_n{N}_{density_type}_{value}_t{trial}.csv
| # Where:
| # - N: node count (100, 500, 1000, 2000, 5000)
| # - density_type: "const" or "scale"
| # - value: probability value (0.05, 0.10, 0.15, 0.20 for const;
| # 0.1, 0.3, 0.5, 0.7 for scale)
| # - trial: trial number (1-5)

```

Directory Structure Explanation

- **Main Project Directory:** Contains all the main scripts and configuration files for the Eulerian path checking project.
- **Data Subdirectories:**
 - data/ - Likely stores data files used in the project

- docker/ - Docker configuration files
 - documentation/ - Project documentation
 - experiments/ - Experimental code or results
 - import/ - Data import utilities
 - ingest/ - Data ingestion scripts for different databases (Memgraph, Neo4j, SQLite)
 - profiles/ - Performance profiling data
 - queries/ - Database query scripts
 - results/ - Output results from experiments
- **Graph Data Directory:** Contains all generated graph files with a specific naming convention for different node counts, density types, probability values, and trial numbers.

B.1 How to reproduce the experiments

To set up the environment and run the experiments:

1. Navigate to the main project directory:

```
cd H:\projects\codes
```

2. Generate graph data (if not already generated):

```
python randomn_graph_gen.py
```

3. Execute the database ingestion and testing scripts:

```
python sql_eulerian_checking.py
python memgraph_eulerian_checking.py
python neo4j_check.py
python eulerian_check_neo4.py
```

4. Analyze the results:

```
python analyze.py
```

5. For Neo4j operations using Docker:

```
./docker-neo4j.sh  
./start-movies-db.sh
```

B.2 File descriptions

randomn_graph_gen.py Generates Erdős–Rényi graphs with various parameters

sql_eulerian_checking.py Performs Eulerian checks using SQL database

memgraph_eulerian_checking.py Performs Eulerian checks using Memgraph

neo4j_check.py Performs Eulerian checks using Neo4j

eulerian_check_neo4.py Additional Neo4j checking script

analyze.py Analyzes and visualizes experiment results

***.csv files** Contain experiment results and generated graph data

***.png and *.pdf files** Visualizations and performance profiles

docker-neo4j.sh, start-movies-db.sh Docker container management scripts

movies.cypher Sample Cypher queries for the movie database

import/ Data import directory for Neo4j

ingest/ Contains ingestion scripts for different databases

B.3 Graph data parameters

The experiments used the following parameters:

- Node counts: $N \in \{100, 500, 1000, 2000, 5000\}$
- Constant densities: $p \in \{0.05, 0.10, 0.15, 0.20\}$
- Scaling densities: $p(N) = N^{-a}$ with $a \in \{0.1, 0.3, 0.5, 0.7\}$
- 5 trials for each parameter combination

Appendix C

Analysis scripts (key snippets)

Below are the essential Python snippets used for computing means and 95% confidence intervals and for generating the diagnostic plots referenced in the main text.

```
# analyze.py (excerpt)
import numpy as np
import scipy.stats as st

def mean_and_95ci(arr):
    a = np.array(arr)
    m = a.mean()
    se = a.std(ddof=1) / np.sqrt(len(a))
    h = se * st.t.ppf(1-0.025, df=len(a)-1)
    return m, h
```

Plotting a diagnostic T vs $|\mathcal{E}|$ (log-log) to show correlation:

```
# plot_time_vs_edges.py (excerpt)
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score
# N_values, edge_counts, means loaded from CSV...
plt.errorbar(edge_counts, means, yerr=cis, marker='o', linestyle
             ='-')

plt.xscale('log')
plt.yscale('log')
plt.xlabel('Edge count  $|\mathcal{E}|$ ')
plt.ylabel('Mean Eulerian-check time (s)')
```

```
plt.title('Time vs edge-count (log-log)')
plt.grid(True, which='both', ls='--', lw=0.5)
plt.savefig('time_vs_edges.png', dpi=300, bbox_inches='tight')
r2 = r2_score(np.log(edge_counts), np.log(means))
print("log-log R^2:", r2)
```

Appendix D

Representative PROFILE / EXPLAIN snapshots

This appendix contains representative planner outputs collected during experiments. A short excerpt is reproduced here for convenience.

--- neo4j_varlength_profile.txt (excerpt) ---

Operator	Rows	DB Hits	Memory
ProduceResults	1	0	0B
EagerAggregation	1	0	32B
VarLengthExpand(All)	643870	693546	192B
NodeIndexSeek	1	2	376B

Note: VarLengthExpand produces the dominant intermediate rows, explaining the path-explosion behaviour documented in Section 4.4.

Appendix E

Full query texts

All canonical queries (Cypher, SQL, Python) are included in the project directory. Below is a short example of the Eulerian aggregation formulation in SQL:

```
-- SQL: aggregation-based Eulerian check
WITH degs AS (
  SELECT start AS v, COUNT(*) AS deg
  FROM edges
  GROUP BY start
)
SELECT SUM((deg % 2)) AS odd_count FROM degs;
-- Then check connectivity separately with a recursive CTE
```


Bibliography

- Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan L. Reutter, and Domagoj Vrgoč. Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5):68:1–68:40, 2017. doi: 10.1145/3104031.
- Marcelo Arenas, Sebastián Conca, and Jorge Pérez. Counting beyond a yottabyte, or how sparql 1.1 property paths will prevent adoption of the standard. In *Proceedings of the 21st International World Wide Web Conference (WWW)*, pages 629–638, 2012. doi: 10.1145/2187836.2187922.
- Guillaume Bagan, Angela Bonifati, and Benoît Groz. A trichotomy for regular simple path queries on graphs. In *Proceedings of the ACM Symposium on Principles of Database Systems (PODS)*, pages 261–272, 2013. doi: 10.1145/2463664.2467795. URL <https://arxiv.org/abs/1212.6857>. accessed: 2025-08-27.
- Katrin Casel, Markus Löffler, et al. Fine-grained complexity of regular path queries. arXiv preprint, 2021. URL <https://arxiv.org/abs/2101.01945>. accessed: 2025-08-27.
- Mariano P. Consens and Alberto O. Mendelzon. Graphlog: A visual formalism for real life recursion. In *Proceedings of the ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS)*, pages 404–416, 1990. doi: 10.1145/298514.298591.
- Isabel F. Cruz, Alberto O. Mendelzon, and Peter T. Wood. A graphical query language supporting recursion. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 323–330, 1987. doi: 10.1145/38713.38749.

- Alin Deutsch, Yu Xu, Mingxi Wu, and Victor E. Lee. Aggregation support for modern graph analytics in tigergraph. In *Proceedings of the 2020 International Conference on Management of Data (SIGMOD)*, pages 377–392, 2020. doi: 10.1145/3318464.3386144.
- Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat-Pérez, Minh-Duc Pham, and Peter Boncz. The ldbc social network benchmark: Interactive workload. In *Proceedings of the 2015 International Conference on Management of Data (SIGMOD)*, pages 619–630, 2015. doi: 10.1145/2723372.2742786.
- Jing Fan, Adalbert Gerald Soosai Raj, Kesheng Wu, and Philip A. Bohannon. The case against specialized graph analytics engines. In *Proceedings of the Seventh Biennial Conference on Innovative Data Systems Research (CIDR)*, pages 65–72, 2015. URL http://cidrdb.org/cidr2015/Papers/CIDR15_Paper31.pdf. Online Proceedings.
- Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Tobias Lindaaker, Stefan Plantikow, Jacopo Urbani, Wim Martens, Ivan Perez, Luis E. Bonnin, and Dan Olteanu. Cypher: An evolving query language for property graphs. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 1433–1445, 2018. doi: 10.1145/3183713.3190657.
- M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co., 1979. ISBN 0-7167-1045-5.
- Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference (SciPy)*, pages 11–15, 2008.
- Carl Hierholzer. über die möglichkeit, einen linienzug ohne wiederholung und ohne unterbrechung zu umfahren. *Mathematische Annalen*, 6(1):30–32, 1873. Classic note on Eulerian cycles.
- Alexandru Iosup, Tim Hegeman, Wing Lung Ngai, Stijn Heldens, Arnau Prat-Pérez, Thomas Manhardt, Hassan Chafi, Mihai Capota, Narayanan Sundaram, Michael Anderson, Ilie Gabriel Tanase, Yinglong Xia, Lifeng Nai, and Peter

- Boncz. Ldbc graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms. *Proceedings of the VLDB Endowment*, 9(13):1317–1328, 2016. doi: 10.14778/3007263.3007270.
- M. Abo Khamis and coauthors. Output-sensitive evaluation of regular path queries. arXiv preprint, 2024. URL <https://arxiv.org/abs/2412.07729>, accessed: 2025-08-27.
- Petri Kotiranta, Marko Junkkari, and Jyrki Nummenmaa. Performance of graph and relational databases in complex queries. *Applied Sciences*, 12(13):6674, 2022. doi: 10.3390/app12136674.
- Leonid Libkin, Wim Martens, and Domagoj Vrgoč. Querying graphs with data. *Journal of the ACM*, 63(2):14:1–14:53, 2016. doi: 10.1145/2850413.
- Matteo Lissandrini, Martin Brugnara, and Yannis Velegrakis. Beyond macrobenchmarks: Microbenchmark-based graph database evaluation. *Proceedings of the VLDB Endowment*, 12(4):390–403, 2019. doi: 10.14778/3297753.3297759.
- Katja Losemann and Wim Martens. The complexity of regular expressions and property paths in sparql. *ACM Transactions on Database Systems*, 38(4):24, 2013. doi: 10.1145/2494529.
- Wim Martens, Matthias Niewerth, and Tina Popp. A trichotomy for regular trail queries. *Logical Methods in Computer Science*, 19(4):20, 2023. doi: 10.46298/LMCS-19(4:20)2023.
- Alberto O. Mendelzon and Peter T. Wood. Finding regular simple paths in graph databases. In *Proceedings of the 15th International Conference on Very Large Data Bases (VLDB)*, pages 185–193, 1989. URL <http://www.vldb.org/conf/1989/P185.PDF>.
- Mark Needham. neo4j/cypher: Getting the hang of the with statement, 2013. URL <https://www.markhneedham.com/blog/2013/03/20/neo4jcypher-getting-the-hang-of-the-with-statement/>. Blog post; accessed: 2025-08-27.

- Neo4j Documentation. List functions — reduce() (cypher manual), 2024. URL <https://neo4j.com/docs/cypher-manual/current/functions/list/>. Neo4j Cypher manual; accessed: 2025-08-27.
- Frank Neven, Pablo Barceló, and Domagoj Vrgoč. Regular queries on graph databases. *Theoretical Computer Science*, 673:31–83, 2017. doi: 10.1016/j.tcs.2016.12.025.
- Anil Pacaci, Alice Zhou, Jimmy Lin, and M. Tamer Özsu. Do we need specialized graph databases?: Benchmarking real-time social network applications. In *Proceedings of the Workshop on Graph Data Management Experiences and Systems (GRADES)*, pages 1–5, 2017. doi: 10.1145/3131728.3131729.
- Mark Raasveldt and Hannes Mühleisen. Duckdb: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 1981–1985, 2019. doi: 10.1145/3299869.3320212.
- Jim Webber, Emil Eifrem, and Ian Robinson. Graph databases. In *O’Reilly Media, Inc.*, 2013.